UPSKILL CAMPUS (USC) & UCT INTERNSHIP

Industrial Internship Report on

"Smart City Traffic Forecaster & FarmEye – AI Crop & Weed Detector"

Prepared by

Kadingela Ramya

B.Tech – CSE (AI & ML)

Siddhartha Institute of Technology and Sciences

UCT Internship – 2026

GitHub: github.com/Kadingela-Ramya

LinkedIn: linkedin.com/in/ramya-kadingela-21606139b

Executive Summary

This report covers the Industrial Internship provided by Upskill Campus (USC) and UniConverge Technologies Pvt Ltd (UCT). Over 6 weeks, I completed two AI/ML projects: (1) Smart City Traffic Forecaster — a Flask web application using a trained LSTM neural network to predict hourly vehicle counts at city junctions up to 12 hours ahead; and (2) FarmEye — a computer vision application powered by YOLOv8 that detects crops and weeds in farm field images and provides a field health assessment. Both projects are deployed on GitHub, built end-to-end with real datasets, and demonstrate how machine learning can solve practical problems in smart city management and precision agriculture.

Preface

This report documents 6 weeks of industrial internship work with Upskill Campus (USC) and UniConverge Technologies Pvt Ltd (UCT) in 2026. I designed and built two complete AI/ML projects from scratch — each addressing a real-world problem, trained on real datasets, and delivered as a functional web application.

The internship bridged the gap between academic learning and industrial application. Rather than toy problems, I was expected to understand a domain, choose a dataset, train a model, handle deployment challenges, and build a user interface — all independently.

Project 1 — FarmEye — applies YOLOv8 object detection to agricultural field images. Project 2 — Smart City Traffic Forecaster — trains an LSTM on 20 months of junction traffic data to predict the next 12 hours of vehicle counts.

I sincerely thank the Upskill Campus team for organizing this program, and UCT for providing the industry context. This experience pushed me beyond notebooks into real deployment constraints — library conflicts, model serialization, environment differences.

To peers and juniors: commit fully. The problems are real, the deadlines firm, and learning is proportional to effort.

Introduction

About UniConverge Technologies Pvt Ltd (UCT)

UCT is a digital transformation company established in 2013, providing industrial solutions with focus on sustainability and ROI. It works across IoT, Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies, Java Full Stack, and Python.

- UCT Insight — IoT platform (Java/ReactJS) for dashboard, analytics, alerting, and third-party integration.
- Factory Watch — Smart factory platform for OEE monitoring, predictive maintenance, and digital twin capabilities.
- LoRaWAN Solutions — Agritech, Smart Cities, and metering applications.
- Predictive Maintenance — Industrial machine health monitoring using Embedded Systems, IoT, and ML.

About Upskill Campus (USC)

Upskill Campus is a career development platform delivering personalized executive coaching in an affordable, scalable way. In collaboration with UCT and The IoT Academy, USC facilitates end-to-end internship execution. Website: https://www.upskillcampus.com/

Objectives of This Internship

- Gain practical experience applying AI/ML to real industrial problems.
- Design and implement end-to-end solutions — from data to deployed application.
- Improve job prospects through demonstrable, GitHub-hosted project work.
- Develop problem-solving, technical writing, and independent learning skills.

Glossary

Term	**Acronym / Definition**
Long Short-Term Memory	LSTM — recurrent neural network suited to time-series forecasting
You Only Look Once (v8)	YOLOv8 — real-time object detection model by Ultralytics
Mean Absolute Error	MAE — average absolute difference between predictions and actual values
Flask	Lightweight Python web framework for serving ML model predictions
Streamlit	Python-based framework for building interactive data apps quickly

Problem Statement

Project 1: FarmEye – Crop & Weed Detection

Farmers spray herbicides uniformly across fields without knowing the actual distribution of weeds. This wastes chemicals, increases costs, and causes environmental harm. A low-cost, image-based tool that identifies weed locations would enable precision spraying and reduce overall herbicide usage.

Project 2: Smart City Traffic Forecaster

Urban traffic management is largely reactive — signals adjust only after congestion builds up. Cities need predictive tools that anticipate traffic peaks at specific junctions hours in advance, so signal timings can be pre-optimized. The challenge: accurately forecast hourly vehicle counts at multiple junctions using historical data.

Existing and Proposed Solutions

Existing Solutions and Limitations

Traffic systems rely on fixed-cycle signals or sensor-triggered reactive systems — they cannot anticipate demand spikes. Weed management relies on manual scouting or blanket spraying — expensive and imprecise. Existing ML solutions are often proprietary, expensive, and inaccessible to city planners or small farmers.

Proposed Solutions

- FarmEye: YOLOv8 trained on the Roboflow CropAndWeed dataset (~2000 annotated images). Identifies crop vs. weed bounding boxes and computes weed density % for field health assessment.
- Smart City Traffic Forecaster: Two-layer LSTM trained on 48,120 hourly records from 4 junctions (Kaggle Traffic Flow dataset). Predicts next 12 hours of vehicle counts at any selected junction.

Code Submission (GitHub Links)

Project 1 – FarmEye:

```
https://github.com/Kadingela-Ramya/FarmEye-AI
```

Project 2 – Smart City Traffic Forecaster:

```
https://github.com/Kadingela-Ramya/Smart_City_Traffic_Forecaster
```

Report Submission (GitHub Link)

Update the link below once report is pushed to Github

This is my github repository link of both the projects

https://github.com/Kadingela-Ramya/uct-internship-report

Proposed Design / Model

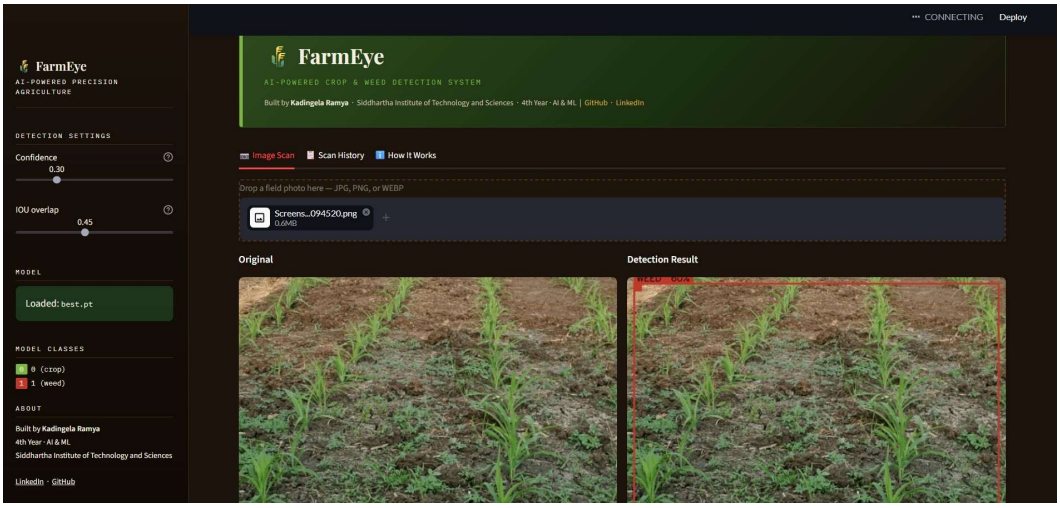
Project 1: FarmEye – System Design

High-Level Flow:

- User uploads a field image (JPG/PNG) via the Streamlit interface
- OpenCV preprocesses the image and passes it to the YOLOv8 model (best.pt)
- Model outputs bounding boxes labelled 'crop' (green) or 'weed' (red)
- Weed density (%) computed as: weed_boxes ÷ total_boxes × 100
- Field health assessment generated based on weed density thresholds
- User can download the annotated image; session scan log is maintained

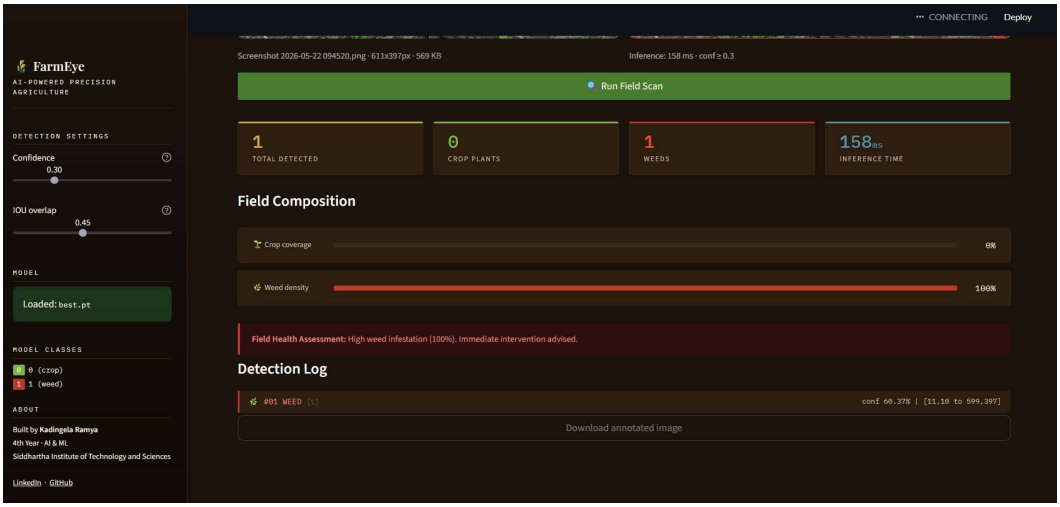
Component	Detail
Base Model	YOLOv8n (nano variant, fastest inference)
Training Platform	Google Colab (free T4 GPU)
Dataset	CropAndWeed from Roboflow (~2,000 annotated images)
Training Duration	~20–30 minutes on T4 GPU
Output	Bounding boxes labelled: crop / weed
Model File	best.pt (placed in models/ folder)

Figure 1: FarmEye – Detection Result



Original field image (left) vs YOLOv8 detection result showing weed bounding box in red (right)

Figure 2: FarmEye – Field Health Assessment



Field composition dashboard: 0% crop coverage, 100% weed density — High infestation alert with detection log

Project 2: Smart City Traffic Forecaster – System Design

High-Level Flow:

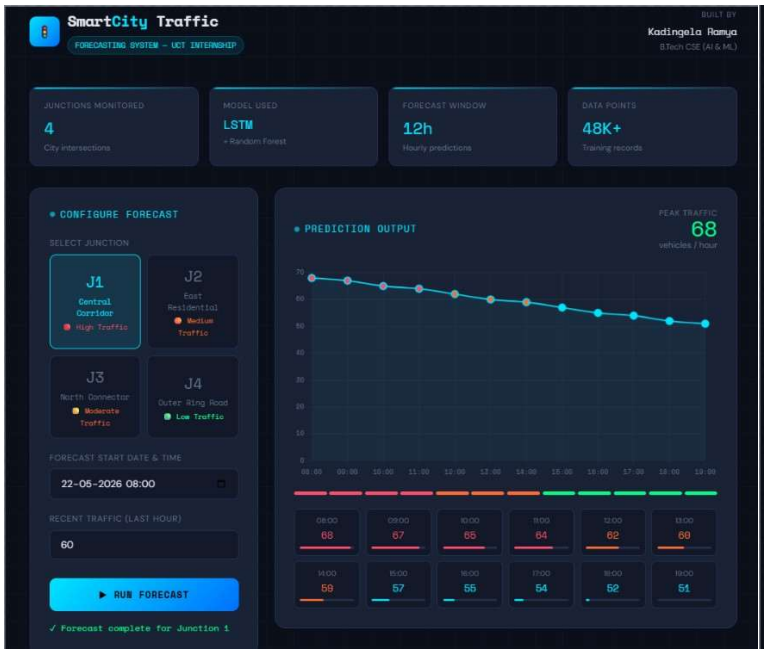
- User selects a junction (J1–J4) and forecast start time via the Flask web interface
- App loads pre-trained LSTM weights (lstm_weights.weights.h5) and scalers from disk
- Input features: hour, day of week, weekend flag, week of year, lag values
- Features scaled using scaler_X.pkl; LSTM predicts next 12 hourly vehicle counts
- Predictions inverse-transformed via scaler_y.pkl and shown as chart + hourly cards

LSTM Architecture:

Layer	**Configuration**
LSTM Layer 1	64 units, return sequences = True
Dropout 1	20% dropout
LSTM Layer 2	32 units
Dropout 2	20% dropout
Dense Layer	16 units, ReLU activation
Output Layer	1 unit, linear activation (vehicle count)

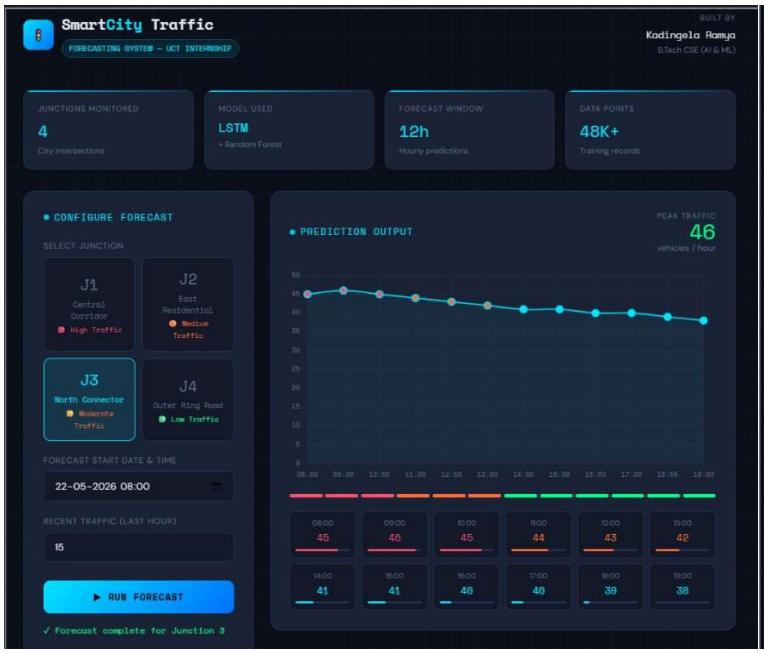
Dataset Attribute	**Value**
Source	Kaggle — Traffic Flow Forecasting dataset
Period Covered	November 2015 – June 2017 (~20 months)
Total Records	48,120 hourly entries
Junctions	4 anonymous city junctions
Junction 1 (Central Corridor)	Avg 45 vehicles/hour — High Traffic
Junction 4 (Outer Ring Road)	Avg 7 vehicles/hour — Low Traffic

Figure 3: Smart City Traffic Forecaster – All 4 Junctions



J1 – Central Corridor (Peak: 68 veh/hr)

J2 – East Residential (Peak: 47 veh/hr)



J3 – North Connector (Peak: 46 veh/hr)

J4 – Outer Ring Road (Peak: 51 veh/hr)

Performance Test

Project 1: FarmEye – Test Cases

Test Case	Input	Expected	Observed
Dense canopy field	High-res crops + weeds	Multiple labelled boxes	Good — individual plant boxes
Bare-soil seedlings	Seedling-stage photo	Detect small plants	Reduced accuracy on small objects
Wide aerial shot	Far-distance crop rows	Row-level detection	Sometimes 1 large box (known limitation)
Empty field	Bare soil image	No detections	Correctly returns 0 detections

Project 2: Traffic Forecaster – Performance Outcome

Metric	Value
Train/Test Split	80% training, 20% test
Mean Absolute Error (MAE)	~3.24 vehicles/hour
Practical interpretation	Predictions within 3–4 vehicles of actual count
Relative error (J1 avg 45 veh/hr)	~7% — acceptable for a planning tool
FarmEye inference time	158 ms per image at confidence ≥ 0.3

The LSTM achieved MAE of ~3.24 vehicles/hour. FarmEye runs inference in 158 ms per image, fast enough for practical field use. Both models perform reliably on their primary use cases, with known limitations documented in the READMEs.

My Learnings

This internship changed how I think about machine learning. Before it, I thought of ML as model training — fitting curves to data. After it, I understand that training is only about 40% of the real work.

Technical Learnings:

- Model serialization & portability: Keras version conflict between Google Colab and local machine forced me to save only weights (.weights.h5) and rebuild the architecture from code — a critical real-world deployment skill.
- Dependency management: Managing numpy<2 alongside TensorFlow, Flask, joblib, and scikit-learn taught me that requirements.txt is not a formality — version conflicts silently break inference.
- Data engineering for time series: Building LSTM input features (lag values, cyclical time encodings) required structuring sequential data into supervised learning windows.
- YOLOv8 training pipeline: Using Roboflow for dataset management, annotation, and YOLO-format export was a new end-to-end workflow learned from scratch.
- Web deployment: Wrapping ML models in Flask and Streamlit interfaces showed how much work goes into making a model accessible to non-technical users.

Soft Skills:

- Working independently with fixed deadlines forced me to prioritize ruthlessly and ship rather than perfect.
- Writing READMEs that a stranger can follow improved my technical communication significantly.
- Understanding that an honest limitations section makes a project more credible, not less.

Future Work Scope

FarmEye

- Retrain on a larger, more diverse dataset with tighter annotations — especially seedling-stage and varying-altitude images.
- Upgrade from YOLOv8n to YOLOv8s or YOLOv8m for improved accuracy on complex scenes.
- Add GPS metadata support — overlay detections on a field map so farmers see weed hotspots geographically.
- Integrate spray recommendation: compute minimum coverage area needing herbicide treatment.

Smart City Traffic Forecaster

- Expand to multi-junction joint forecasting using a graph neural network — congestion at one junction affects neighbors.
- Incorporate external signals: weather data, events calendar, and school/holiday schedules as features.
- Build a live city dashboard with a map view showing predicted congestion color-coded by junction.
- Explore Transformer-based models (e.g., Temporal Fusion Transformer) as an alternative to LSTM.

General

- Package both apps as Docker containers for reproducible deployment across any machine.
- Write automated test suites for prediction pipelines to catch regressions when model files are updated.

References

[1] Kaggle – Traffic Flow Forecasting Dataset: https://www.kaggle.com/datasets

[2] Roboflow CropAndWeed Dataset: https://roboflow.com/

[3] Ultralytics YOLOv8 Documentation: https://docs.ultralytics.com/

[4] TensorFlow/Keras LSTM Docs: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM

[5] Flask Documentation: https://flask.palletsprojects.com/

[6] Streamlit Documentation: https://docs.streamlit.io/